

裸机例程 Makefile 详解

正点原子 i.MX6ULL · 01_leds / 02_ledc / 03_ledc_stm32 / 04_ledc_sdk

从单文件汇编到工程化 SDK —— Makefile 进化全解析

第一章 四个例程 Makefile 演进总览

这四个裸机例程的 Makefile 展示了从最简单的单文件编译到工程化 SDK 风格的完整演进过程。理解这条进化路线，是掌握嵌入式裸机 Makefile 的最快路径。

例程	源文件构成	Makefile 特点	阶段
01_leds	单个汇编文件	无变量、无模式规则、手写 每条命令	入门
02_ledc	汇编 + C 多文件	引入 objs 变量、模式规则 %.o:%.c	进阶
03_ledc_stm32	汇编 + C (STM32 风格)	结构与 02 相同，源码改为 STM32 寄存器操作风格	巩固
04_ledc_sdk	汇编 + C (SDK 风格)	CROSS_COMPILE / NAME 变量，完全参数化	工程化

- 关键进化点：**
- ① 01 → 02：引入模式规则（%.o:%.c），不用为每个文件单独写规则
 - ② 02 → 04：引入变量参数化（CROSS_COMPILE、NAME），换芯片/换项目名只改变量

第二章 01_leds — 最简单的裸机 Makefile

第一个裸机例程只有一个汇编源文件 led.s，Makefile

把每一步编译命令都明确写出，没有任何变量和自动化规则，适合初学者逐行理解。

2.1 Makefile 原文

```
led.bin:led.s
arm-linux-gnueabi-gcc -g -c led.s -o led.o
arm-linux-gnueabi-ld -Ttext 0X87800000 led.o -o led.elf
arm-linux-gnueabi-objcopy -O binary -S -g led.elf led.bin
arm-linux-gnueabi-objdump -D led.elf > led.dis
clean:
rm -rf *.o led.bin led.elf led.dis
```

2.2 逐行详解

语法 / 变量	含义与说明
led.bin : led.s	规则格式：目标：依赖 led.bin 是最终目标，led.s 是依赖文件。 只要 led.s 比 led.bin 新（或 led.bin 不存在），下面的命令就会执行。
gcc -g -c led.s -o led.o	汇编→目标文件 -g：保留调试符号信息 -c：只编译不链接，生成 .o 目标文件 led.s → led.o
ld -Ttext 0X87800000 led.o -o led.elf	链接→ELF 文件 -Ttext 0X87800000：设置代码段起始地址为 0x87800000 这是 i.MX6ULL DDR 的加载地址，BootROM 会把程序拷贝到这里执行 led.o → led.elf
objcopy -O binary -S -g led.elf led.bin	ELF→纯二进制 -O binary：输出裸二进制格式（去掉 ELF 头） -S：去掉符号表和重定位信息 -g：去掉调试信息 ELF 有格式头无法直接烧录，.bin 才是烧进 SD 卡的文件
objdump -D led.elf > led.dis	生成反汇编文件（调试用） -D：对所有段进行反汇编 > led.dis：将输出重定向到文件，方便查看汇编指令与地址的对应关系
clean:	伪目标，无依赖 make clean 时执行，删除所有编译产物，下次 make 时全部重新编译

2.3 编译产物说明

文件	类型	用途
led.o	目标文件	中间产物，可删除
led.elf	ELF 可执行	含调试信息，用于 objdump / GDB
led.bin	纯二进制	烧录到 SD 卡的文件
led.dis	反汇编文本	调试时查看机器码与汇编对应关系

为什么地址是 0x87800000?

i.MX6ULL 的 DDR 起始地址是 0x80000000。正点原子课程将程序加载到偏移 0x7800000 处，避开 U-Boot 和内核的常用区域，是裸机实验的约定地址。

第三章 02_ledc — 多文件编译与模式规则

02_ledc 新增了 C 源文件 main.c 和链接脚本 imx6ul.lids, Makefile 引入了变量和模式规则, 不需要为每个源文件单独写编译命令。

3.1 Makefile 原文

```
objs := start.o main.o

ledc.bin:$(objs)
arm-linux-gnueabi-hf-ld -T imx6ul.lids -o ledc.elf $^
arm-linux-gnueabi-hf-objcopy -O binary -S ledc.elf $@
arm-linux-gnueabi-hf-objdump -D -m arm ledc.elf > ledc.dis

%.o:%.s
arm-linux-gnueabi-hf-gcc -Wall -nostdlib -c -O2 -o $@ $<

%.o:%.S
arm-linux-gnueabi-hf-gcc -Wall -nostdlib -c -O2 -o $@ $<

%.o:%.c
arm-linux-gnueabi-hf-gcc -Wall -nostdlib -c -O2 -o $@ $<

clean:
rm -rf *.o ledc.bin ledc.elf ledc.dis
```

3.2 新增语法详解

语法 / 变量	含义与说明
objs := start.o main.o	变量定义: 把所有目标文件列表存入 objs 变量。 := 立即展开赋值。后续用 \$(objs) 引用, 增减源文件只改这一行。
ledc.bin : \$(objs)	依赖展开为 start.o main.o, Make 先确保这两个 .o 存在再执行链接。
-T imx6ul.lids	链接脚本: 替代了 01_leds 的 -T text 简单写法。 链接脚本可以精确控制每个段 (.text/.data/.bss) 的起始地址和排列顺序, 适合多文件工程。
\$^	自动变量: 展开为所有依赖文件, 即 start.o main.o。 避免手写依赖列表, 添加新文件只需改 objs 变量。
\$@	自动变量: 展开为当前规则的目标文件名, 此处为 ledc.bin。
\$<	自动变量: 展开为第一个依赖文件名, 在模式规则中即当前要编译的那个源文件。

语法 / 变量	含义与说明
%o : %c (模式规则)	% 是通配符，匹配任意字符串。 %.o:%.c 表示"所有 .o 文件都可以由同名 .c 文件编译而来"。 有了这条规则，无论有多少 .c 文件，Make 都知道怎么编译，不用逐一列出。
-Wall	开启所有常见警告，帮助发现代码潜在问题。
-nostdlib	不链接标准 C 库。裸机程序没有操作系统提供 libc，必须加此选项，否则链接器会尝试找 _start 等 libc 符号而报错。
-O2	二级优化：编译器对代码进行速度优化，适合裸机性能要求。
-m arm	告诉 objdump 按 ARM 指令集反汇编（而非 Thumb），确保显示正确。

三条模式规则的区别：

%.o:%.s — 处理小写 .s（纯汇编，不经预处理器）

%.o:%.S — 处理大写 .S（汇编 + 预处理，可以用 #include / #define）

%.o:%.c — 处理 C 源文件

start.S 通常用大写，因为需要 #include 头文件。

3.3 imx6ul.lds 链接脚本原文

Makefile 中的 -Tmx6ul.lds

指定了链接脚本文件。链接脚本控制最终二进制文件中各段（Section）的内存布局。

SECTIONS{
. = 0X87800000;
.text :
{
start.o
main.o
*(.text)
}
.rodata ALIGN(4) : {*(.rodata*)}
.data ALIGN(4) : {*(.data) }
__bss_start = .;
.bss ALIGN(4) : {*(.bss) *(COMMON) }
__bss_end = .;
}

3.4 链接脚本逐行详解

语法 / 变量	含义与说明
SECTIONS { }	GNU ld 链接脚本的核心关键字，花括号内定义所有输出段的布局。
. = 0x87800000	位置计数器 (Location Counter) 赋值。 . 代表"当前地址"，设为 0x87800000 后，后续所有段都从这个地址开始依次排列。 i.MX6ULL DDR 起始地址 0x80000000，0x87800000 是裸机实验的约定加载地址。
.text : { start.o main.o *(.text) }	代码段 (可执行指令)。 花括号内控制该段内各目标文件的排列顺序： ① start.o 排第一：CPU 上电后跳到 0x87800000，必须最先执行启动代码 ② main.o 排第二：紧接着是 main 函数 ③ *(.text)：其余所有目标文件的 .text 段追加到后面
.rodata ALIGN(4) : { *(.rodata*) }	只读数据段：字符串字面量、const 全局数组等。 ALIGN(4) 表示此段起始地址按 4 字节对齐，确保 ARM 访问不产生对齐异常。
.data ALIGN(4) : { *(.data) }	已初始化数据段：有初始值的全局变量 / 静态变量。 如 int a = 5; 会放在这里，初始值存在 .bin 文件中。
__bss_start = .	定义符号：把当前地址赋给符号 __bss_start。 这个符号会被导出，start.S 启动代码可以用它定位 BSS 段起始地址，用于启动时将 BSS 段清零 (C 标准要求未初始化全局变量为 0)。
.bss ALIGN(4) : { *(.bss) *(COMMON) }	未初始化数据段：没有初始值的全局变量 / 静态变量。 int b; 会放在这里，不占 .bin 文件空间，运行前由启动代码清零。 *(COMMON) 收集未声明为 static 的公共符号。
__bss_end = .	BSS 段结束地址符号，配合 __bss_start 让启动代码知道清零范围。

为什么 start.o 必须排在 .text 段第一位?

CPU 上电复位后 PC 直接跳到 0x87800000 开始执行。

如果 start.o 不在第一位，CPU 会执行到其他函数的指令，立即崩溃。

链接脚本的核心价值之一就是保证启动代码在最低地址。

第四章 03_ledc_stm32 — STM32 风格寄存器操作

03_ledc_stm32 的 Makefile 与 02_ledc 完全相同，区别在于 C 源码：引入了类似 STM32 HAL 库的寄存器结构体封装方式（MCIMX6Y2.h / fsl_iomuxc.h），对比纯寄存器操作理解外设驱动抽象。

4.1 Makefile 原文

```
objs := start.o main.o

ledc.bin:$(objs)
arm-linux-gnueabi-hf-ld -Tmx6ul.lds -o ledc.elf $^
arm-linux-gnueabi-hf-objcopy -O binary -S ledc.elf $@
arm-linux-gnueabi-hf-objdump -D -m arm ledc.elf > ledc.dis

%.o:%.s
arm-linux-gnueabi-hf-gcc -Wall -nostdlib -c -O2 -o $@ $<

%.o:%.S
arm-linux-gnueabi-hf-gcc -Wall -nostdlib -c -O2 -o $@ $<

%.o:%.c
arm-linux-gnueabi-hf-gcc -Wall -nostdlib -c -O2 -o $@ $<

clean:
rm -rf *.o ledc.bin ledc.elf ledc.dis
```

4.2 与 02_ledc 的对比

对比项	变化	说明
Makefile	完全相同	两个例程的构建方式没有任何区别
start.S	基本相同	汇编启动代码，设置栈指针后跳转到 main
main.c	不同	02_ledc 直接操作寄存器地址（手动 ioremap）； 03_ledc_stm32 使用结构体指针（CCM->CCGR0）访问寄存器，风格类似 STM32 HAL
头文件	新增	引入 MCIMX6Y2.h（寄存器结构体定义）和 fsl_iomuxc.h（IO 复用宏定义）

为什么引入结构体方式？

直接写地址 (*(unsigned int*)0x020C4068)：直观但易出错，可读性差。
结构体方式 (CCM->CCGR0)：寄存器名字即含义，IDE 可自动补全，与 NXP 官方 SDK 风格一致。
Makefile 不变说明：构建工具与源码风格无关，换写法不换 Makefile。

4.3 imx6ul.lds 链接脚本原文

03_ledc_stm32 的链接脚本与 02_ledc 只有一处差异：.text 段中去掉了显式的 main.o 排列，改由 *(.text) 自动收录。

```
SECTIONS{
. = 0X87800000;
.text :
{
start.o
*(.text)
}
.rodata ALIGN(4) : {*(.rodata*)}
.data ALIGN(4) : { *(.data) }
__bss_start = ;
.bss ALIGN(4) : { *(.bss) *(COMMON) }
__bss_end = ;
}
```

4.4 与 02_ledc 链接脚本的差异

.text 段排列	写法	效果
02_ledc	start.o main.o *(.text)	start.o 和 main.o 都显式列出，链接顺序完全确定
03_ledc_stm32	start.o *(.text)	只显式列出 start.o，main.o 由 *(.text) 收录，顺序由链接器决定

两种写法的实际影响：

对于只有 start.o + main.o 两个文件的简单工程，*(.text) 通常也会按链接顺序将 main.o 紧跟 start.o。但文件多时，显式列出关键文件（如 02_ledc 的写法）更安全，能确保初始化代码顺序正确。

第五章 04_ledc_sdk — 工程化 SDK 风格 Makefile

04_ledc_sdk 是这四个例程中 Makefile 最规范版本：引入 CROSS_COMPILE 和 NAME 变量，将交叉编译器前缀和工程名完全参数化，是实际嵌入式项目 Makefile 的标准写法。

5.1 Makefile 原文

```
CROSS_COMPILE ?= arm-linux-gnueabi-
NAME      ?= ledc

CC      := $(CROSS_COMPILE)gcc
LD      := $(CROSS_COMPILE)ld
OBJCOPY := $(CROSS_COMPILE)objcopy
OBJDUMP := $(CROSS_COMPILE)objdump

OBJS := start.o main.o

$(NAME).bin:$(OBJS)
$(LD) -Tmx6ul.lds -o $(NAME).elf $^
$(OBJCOPY) -O binary -S $(NAME).elf $@
$(OBJDUMP) -D -m arm $(NAME).elf > $(NAME).dis

%.o:%.s
$(CC) -Wall -nostdlib -c -O2 -o $@ $<

%.o:%.S
$(CC) -Wall -nostdlib -c -O2 -o $@ $<

%.o:%.c
$(CC) -Wall -nostdlib -c -O2 -o $@ $<

clean:
rm -rf *.o $(NAME).bin $(NAME).elf $(NAME).dis
```

5.2 新增变量详解

语法 / 变量	含义与说明
CROSS_COMPILE ?= arm-linux-gnueabi-	?= 条件赋值：只有当 CROSS_COMPILE 未被定义时才赋此默认值。 这样可以在命令行覆盖： make CROSS_COMPILE=aarch64-linux-gnu- 换芯片架构只需改这一个变量，所有工具（gcc/ld/objcopy）自动跟随变化。
NAME ?= ledc	工程名变量：控制所有输出文件的名字前缀。 \$(NAME).bin / \$(NAME).elf / \$(NAME).dis 随 NAME 自动变化。 make NAME=myapp 即可生成 myapp.bin，无需修改 Makefile 内部。

语法 / 变量	含义与说明
CC := \$(CROSS_COMPILE)gcc	将交叉编译器前缀与工具名拼接。 展开结果：arm-linux-gnueabi-hf-gcc LD / OBJCOPY / OBJDUMP 同理，统一由 CROSS_COMPILE 控制。
\$(NAME).bin : \$(OBJS)	目标文件名由变量决定，比硬编码 ledc.bin 更灵活。

5.3 ?= 与 := 赋值方式对比

符号	名称	特点	典型场景
:=	立即展开赋值	右侧在读取时求值，结果不变	OBJS := start.o main.o
=	延迟展开赋值	右侧每次引用时才求值	很少用于路径/工具
?=	条件赋值	变量未定义时才赋值，可被命令行覆盖	CROSS_COMPILE ?= ...
+=	追加赋值	在已有值后面追加	OBJS += extra.o

5.4 命令行使用示例

```
# 默认编译（使用 ARM 交叉编译器，生成 ledc.bin）
make

# 指定工程名
make NAME=myled

# 切换到 AArch64 编译器
make CROSS_COMPILE=aarch64-linux-gnu-

# 清理
make clean
```

04_ledc_sdk 相比前面例程的核心进步：
前三个例程的工具链名称 arm-linux-gnueabi-hf- 散落在每条命令里，换架构要改很多行。
04_ledc_sdk 把它提取成 CROSS_COMPILE 变量：换芯片只改一个地方，这正是 Linux 内核 Makefile 的设计思路。

5.5 imx6ul.lds 链接脚本原文

04_ledc_sdk 的链接脚本与 02_ledc 完全相同，显式排列 start.o 和 main.o，保证启动代码与主函数的加载顺序。

```
SECTIONS{
```

```
. = 0x87800000;

.text :
{
start.o
main.o

*(.text)
}

.rodata ALIGN(4) : {*(.rodata*)}
.data ALIGN(4) : {*(.data) }

__bss_start = .;

.bss ALIGN(4) : { *(.bss) *(COMMON) }

__bss_end = .;
}
```

5.6 内存布局示意

地址	段	内容	来源
0x87800000	.text	启动代码 + 所有函数指令	start.o → main.o → *.o
紧随 .text	.rodata	字符串字面量、const 数组	4字节对齐
紧随 .rodata	.data	有初始值的全局/静态变量	4字节对齐
紧随 .data (__bss_start)	.bss	无初始值的全局/静态变量 (启动代码负责清零)	4字节对齐 不占 .bin 空间
(__bss_end)	—	栈从这里向下生长 (start.S 设置 SP)	运行时动态使用

__bss_start 和 __bss_end 在 C 代码中的作用：
start.S 中通常有如下清零循环（伪代码）：
addr = __bss_start;
while (addr < __bss_end) { *addr++ = 0; }
C 标准规定未初始化全局变量值为 0，但硬件上电后内存是随机值，必须手动清零。

第六章 横向对比与学习建议

6.1 四个 Makefile 特性对比

特性	01_leds	02_ledc	03_ledc_stm32	04_ledc_sdk
多源文件支持	❑	❑	❑	❑
模式规则 %.o:%.c	❑	❑	❑	❑
链接脚本 .lds	❑	❑	❑	❑
自动变量 \$@ \$< \$^	❑	❑	❑	❑
工具链变量 CC/LD	❑	❑	❑	❑
CROSS_COMPILE 参数	❑	❑	❑	❑
NAME 参数化	❑	❑	❑	❑
命令行覆盖变量	❑	❑	❑	❑

6.2 自动变量速查

语法 / 变量	含义与说明
\$@	当前规则的目标文件名
\$^	当前规则的所有依赖文件（去重）
\$<	当前规则的第一个依赖文件（模式规则中即当前源文件）
\$*	模式规则中 % 匹配到的部分（文件名去掉后缀）

6.3 学习建议

建议按以下顺序阅读和模仿，每写完一个都用 make 实际编译验证：

第一步	读懂 01_leds	逐行理解 gcc / ld / objcopy / objdump 的作用
第二步	改写 02_ledc	自己手写 objs 变量和模式规则，理解 \$@ \$< \$^
第三步	跳过 03_ledc_stm32	它的 Makefile 与 02 完全相同，重点看 C 源码变化
第四步	模仿 04_ledc_sdk	为自己的项目加入 CROSS_COMPILE 和 NAME，形成工程模板

04_ledc_sdk 就是你自己后续裸机项目的 Makefile 模板。
换新项目时：改 NAME 变量（项目名）、改 OBJS 变量（源文件列表），其余不动即可复用。